

Morse Code Converter — Project Report

1. Cover Page

PROJECT TITLE: Morse Code Converter Application

DEVELOPER: Surjithh M,J

DATE OF SUBMISSION: September 28, 2026

REPOSITORY LINK: <https://github.com/surjithh26bhi10007-droid/vityarthi-project>

2. Introduction

Morse code is a long-standing telecommunication method that encodes text characters as standardized sequences of short and long signals (dots . and dashes -). Modern digital communication systems often require simple utility programs to bridge legacy communication schemes with standard digital text formats.

The **Morse Code Converter** is a lightweight Python-based desktop application designed to perform bi-directional translation between alphanumeric English text and Morse code representations[cite: 1]. Operating through an interactive menu system in the command-line interface (CLI), the software provides real-time encoding and decoding capabilities[cite: 1].

3. Problem Statement

Manual encoding and decoding of Morse code is slow, prone to human error, and requires specialized memory of signal mappings. While modern messaging relies on standard ASCII/Unicode strings, educational contexts, ham radio operations, and simple signaling setups often require converting text back and forth to Morse code quickly.

There is a need for a lightweight, dependency-free utility that:

1. Translates plain text into standard Morse code representation accurately[cite: 1].
2. Decodes formatted Morse code signals back into readable English text[cite: 1].
3. Provides an accessible interactive environment without requiring complex software installations or database dependencies[cite: 1].

4. Functional Requirements

The system provides the following functional capabilities:

- **FR-1: Text to Morse Code Encoding**
 - The user must be able to input standard text containing alphanumeric characters[cite: 1].

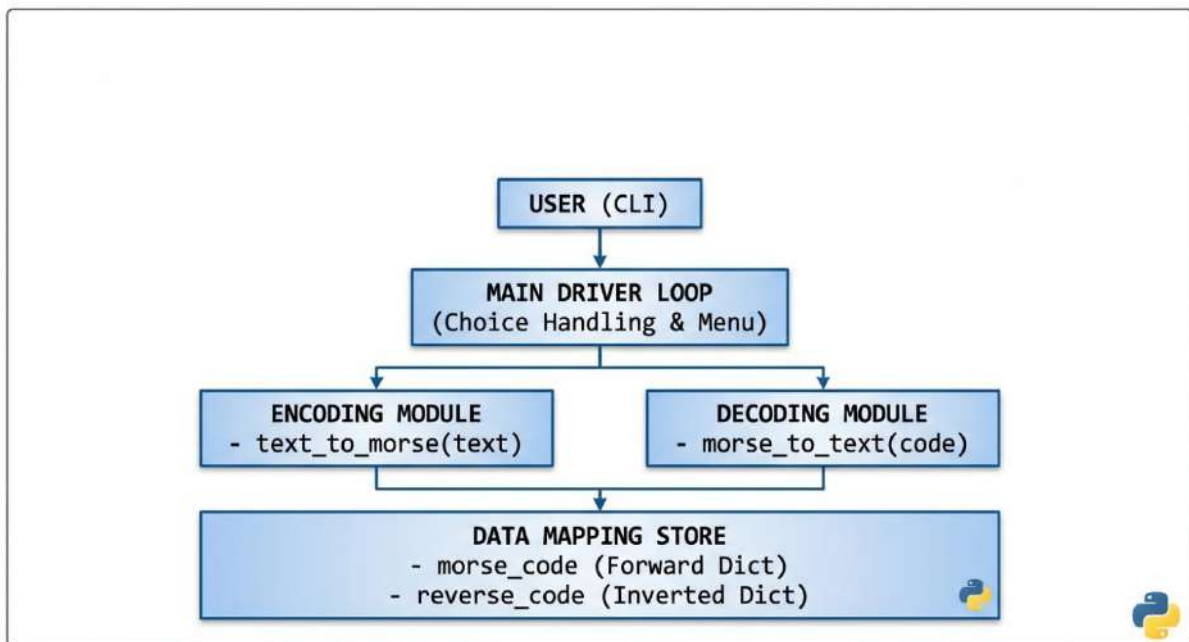
- The system must convert all input characters to uppercase before encoding to ensure case invariance[cite: 1].
- The system must insert a single space between individual encoded characters and double spaces between distinct words[cite: 1].
- **FR-2: Morse Code to Text Decoding**
 - The user must be able to input space-delimited Morse code sequences[cite: 1].
 - The system must parse single-space delimiters as letter separators and double-space delimiters as word separators[cite: 1].
 - The system must reconstruct and return the translated English text string[cite: 1].
- **FR-3: Interactive User Interface**
 - The CLI must present an interactive selection menu with numbered options (1, 2, 3)[cite: 1].
 - The application must validate menu choices and display an error message for invalid entries[cite: 1].
- **FR-4: Session Management**
 - The application must continuously prompt the user until the explicit selection of option 3 (Exit) breaks the execution loop[cite: 1].

5. Non-functional Requirements

- **NFR-1: Performance Efficiency:** The lookup operation for encoding and decoding must execute in $O(1)$ constant time per character using hash-based Python dictionary structures[cite: 1].
- **NFR-2: Portability & Zero External Dependencies:** The application must run on any environment supporting standard Python 3.x without requiring external library installations (pip install)[cite: 1].
- **NFR-3: Usability:** The terminal UI must use visual framing borders to clearly highlight menu options and feedback prompts[cite: 1].
- **NFR-4: Reliability:** The application must handle unknown or non-mapped input gracefully without crashing the main application loop[cite: 1].

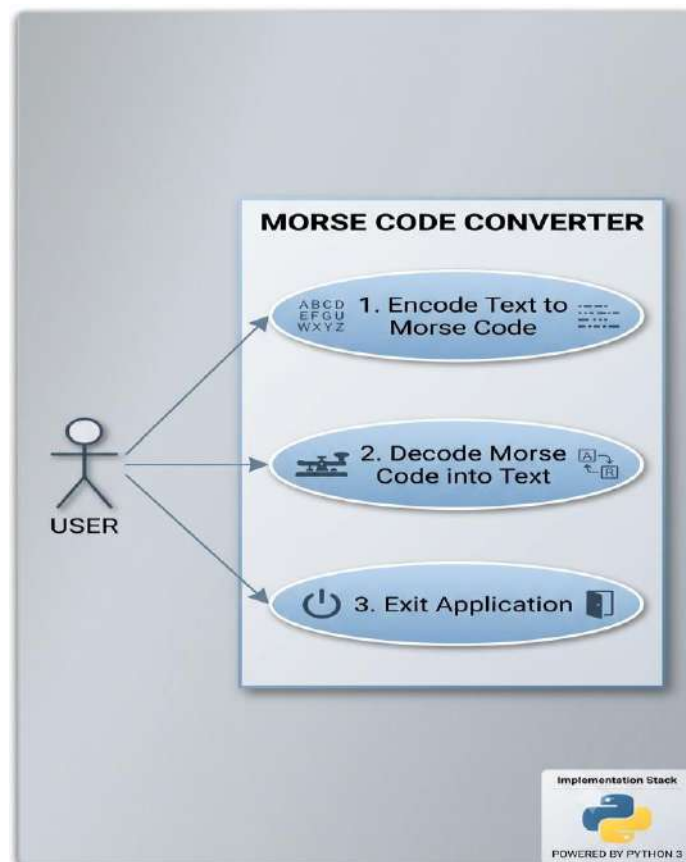
6. System Architecture

The application follows a simple modular architectural structure contained within a executable Python script:

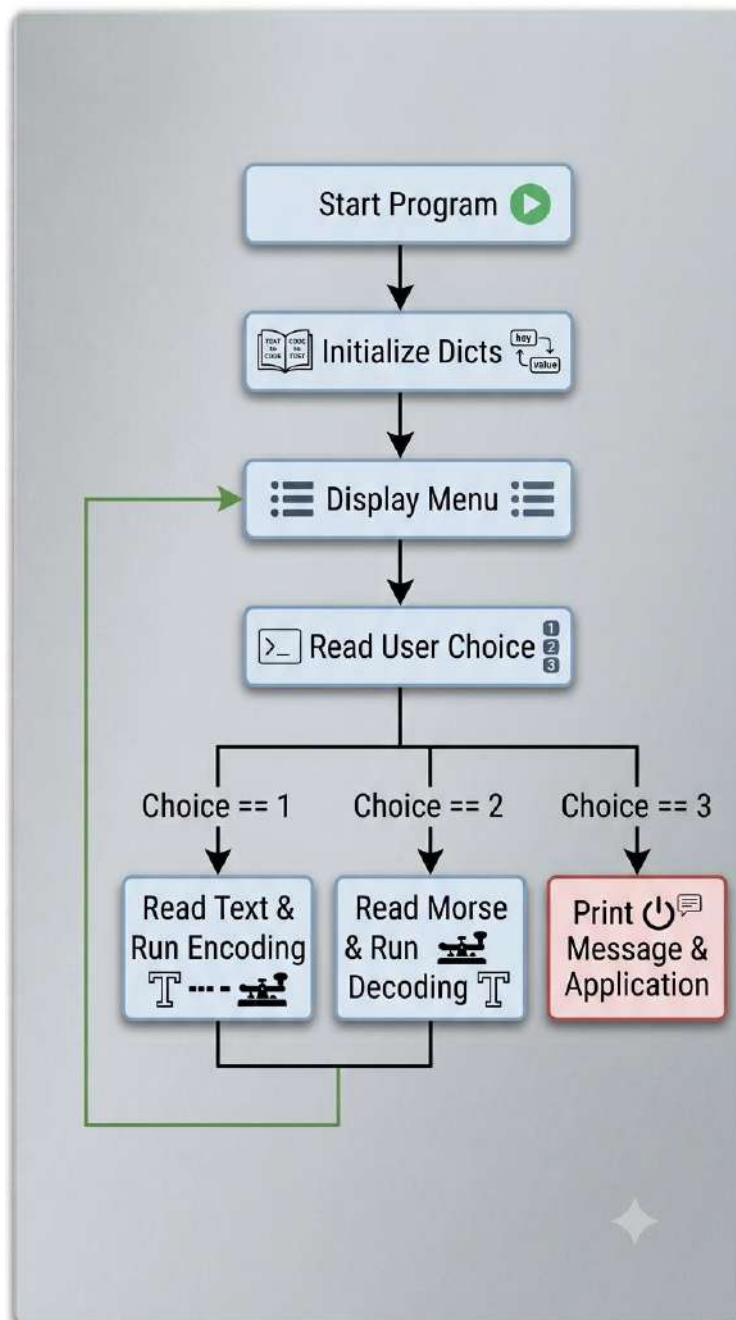


7. Design Diagrams

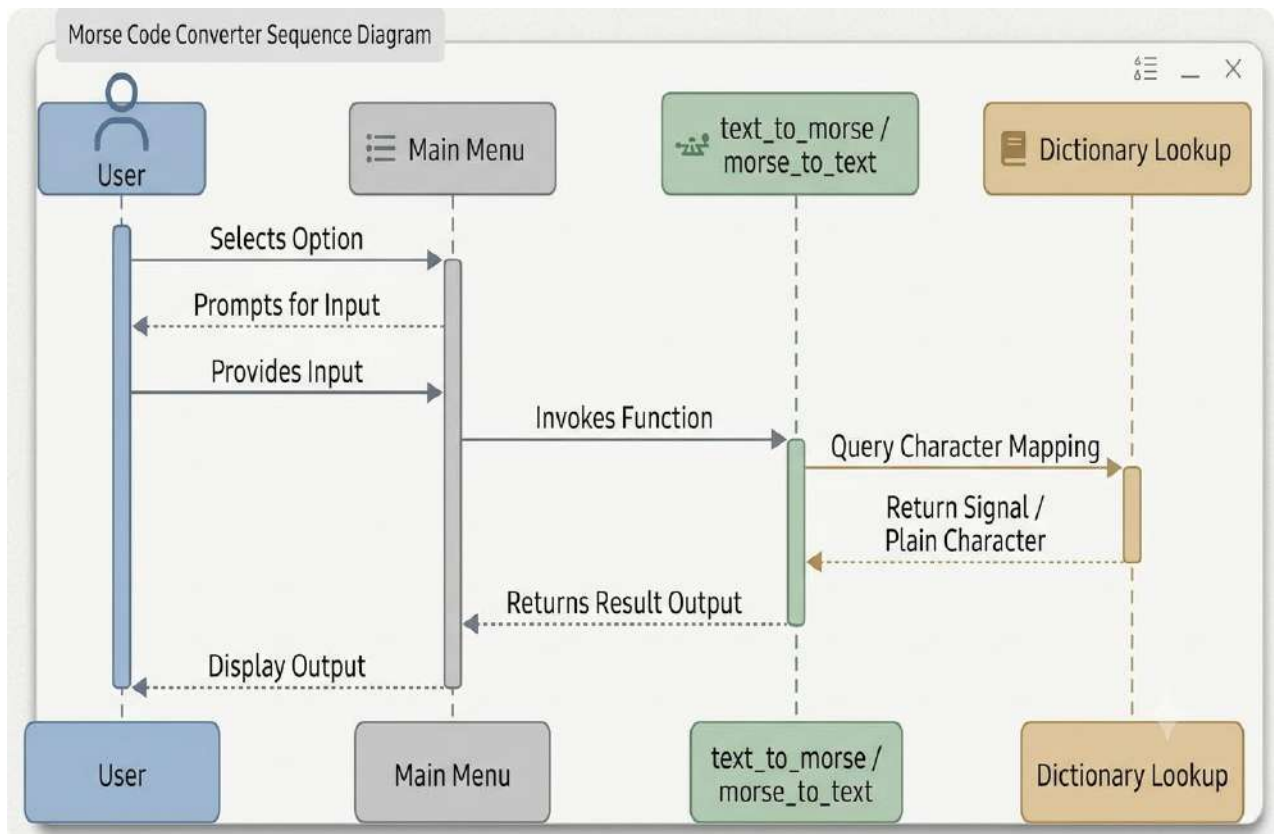
7.1 Use Case Diagram



7.2 Workflow Diagram



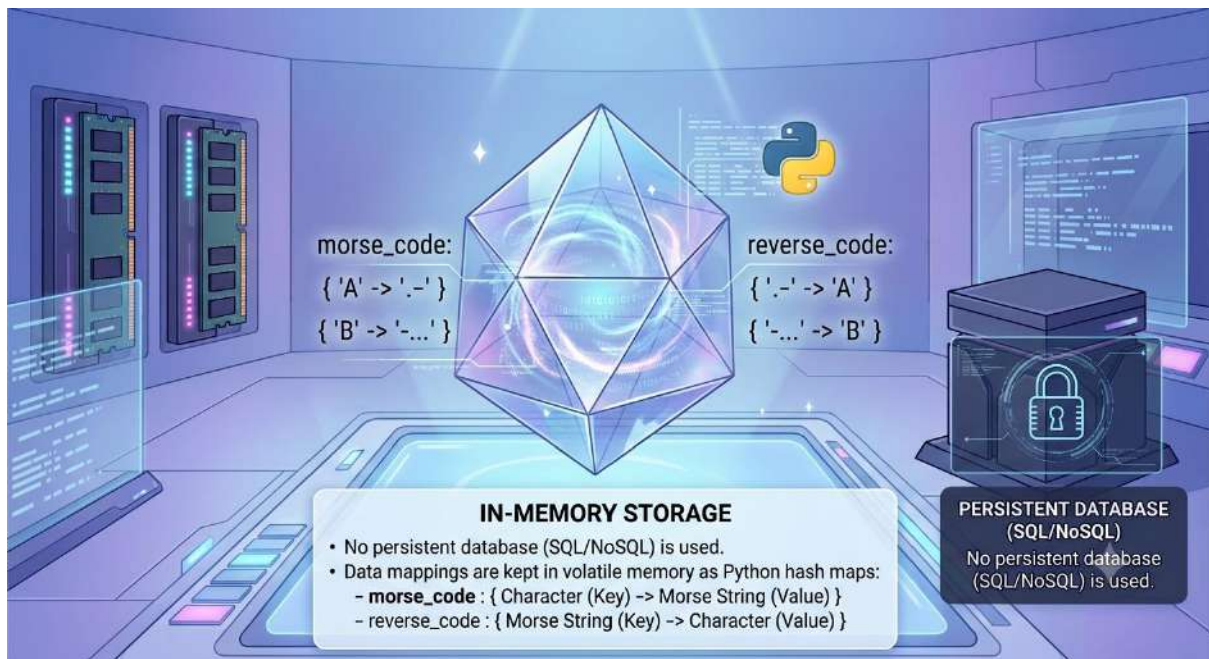
7.3 Sequence Diagram



7.4 Class / Component Diagram



7.5 ER Diagram



8. Design Decisions & Rationale

1. Dynamic Reverse Mapping Generation:

- **Decision:** Rather than hardcoding two separate lookup tables, the inverted dictionary (reverse_code) is generated programmatically at launch using a loop over morse_code[cite: 1].
- **Rationale:** Reduces redundant code, minimizes maintenance overhead, and prevents discrepancies between forward and reverse translation mappings[cite: 1].

2. Choice of Data Structure (Hash Maps):

- **Decision:** Python dictionaries are used to store letter-to-code associations[cite: 1].
- **Rationale:** Dictionaries offer key-value lookup operations in $O(1)$ average time complexity, ensuring fast translation execution.

3. Space Delimiters for Word/Letter Separation:

- **Decision:** Single spaces separate individual Morse letters; double spaces separate complete words[cite: 1].
- **Rationale:** Preserves standard visual notation while providing simple separation logic during parsing operations[cite: 1].

9. Implementation Details

The program is implemented in standard Python 3[cite: 1]. Below is the core source code:

Python

```
morse_code = {
    'A': '-.-', 'B': '-...', 'C': '-.-.', 'D': '-..',
    'E': '.', 'F': '..-.', 'G': '--.', 'H': '....',
    'I': '..', 'J': '.---', 'K': '-.-', 'L': '-.-.',
    'M': '--', 'N': '-.', 'O': '---', 'P': '-.-.',
    'Q': '--.-', 'R': '.-.', 'S': '...', 'T': '-',
    'U': '..-', 'V': '...-', 'W': '-.-', 'X': '-.-.',
    'Y': '-.-.', 'Z': '--..',
    '0': '-----', '1': '.-----', '2': '..---', '3': '...--',
    '4': '....-', '5': '.....', '6': '-.....', '7': '--....',
    '8': '---..', '9': '----.'}

reverse_code = {}

for letter in morse_code:
    reverse_code[morse_code[letter]] = letter

def text_to_morse(text):
    result = ""
    for letter in text.upper():
        if letter in morse_code:
            result = result + morse_code[letter] + " "
        elif letter == " ":
            result = result + " "
    return result
```

```

def morse_to_text(code):
    result = ""
    words = code.split(" ")
    for word in words:
        letters = word.split()
        for letter in letters:
            if letter in reverse_code:
                result = result + reverse_code[letter]
        result = result + " "
    return result

while True:
    print("\n=====|")
    print("    MORSE CODE CONVERTER    |")
    print("=====|")
    print("1. Text to Morse Code    |")
    print("2. Morse Code to Text    |")
    print("3. Exit                  |")
    print("=====|")

    choice = input("Enter your choice as number: ")
    if choice == "1":
        text = input("Enter text: ")
        result = text_to_morse(text)
        print("Morse Code:", result)
    elif choice == "2":
        code = input("Enter Morse Code: ")
        result = morse_to_text(code)
        print("Text:", result)

```

```
elif choice == "3":  
    print("Thank you for using Morse Code Converter!")  
    break  
else:  
    print("Invalid choice!")  
    print("Please enter 1, 2, or 3.")  
``[cite: 1]  
---
```

10. Screenshots / Results

File Edit Shell Debug Options Window Help

```
MORSE CODE CONVERTER
=====
1. Text to Morse Code
2. Morse Code to Text
3. Exit
=====
Enter your choice as number: 1
Enter text: hello google
Morse Code: .... . .-... .-... --- --. --- --- --. .-... .

=====
MORSE CODE CONVERTER
=====
1. Text to Morse Code
2. Morse Code to Text
3. Exit
=====
Enter your choice as number: 1
Enter text: enter
Morse Code: . -. - . .-.

=====
MORSE CODE CONVERTER
=====
1. Text to Morse Code
2. Morse Code to Text
3. Exit
=====
Enter your choice as number: 2
Enter Morse Code: .--.
Text: P

=====
MORSE CODE CONVERTER
=====
1. Text to Morse Code
2. Morse Code to Text
3. Exit
=====
Enter your choice as number: 3
Thank you for using Morse Code Converter!
>>>
```

11. Testing Approach

The system was tested manually using test cases covering functional execution paths:

Test Case ID	Description	Input	Expected Output	Pass/Fail
TC-01	Convert lower-case text	hello	 -.-. -.-. --- [cite: 1]	Pass
TC-02	Convert mixed-case sentence	Hi User	 -.-. -.-. [cite: 1]	Pass
TC-03	Decode valid Morse sequence		HE [cite: 1]	Pass
TC-04	Invalid menu choice handling	5	Invalid choice! Please enter 1, 2, or 3.[cite: 1]	Pass
TC-05	Graceful application exit	3	Exit message and termination[cite: 1]	Pass

12. Challenges Faced

1. Indentation and Scope Bug in Morse-to-Text Conversion:

- *Challenge:* In the original script implementation, the inner letter-parsing loop was incorrectly indented outside the primary word loop[cite: 1]. As a result, multi-word strings failed to process correctly[cite: 1].
- *Resolution:* Corrected block alignment so that letter extraction happens within each word unit iteration[cite: 1].

2. Handling Unsupported Punctuation:

- *Challenge:* Non-alphanumeric symbols (e.g., ,, !, ?) were silently dropped during translation[cite: 1].
- *Resolution:* Identified missing symbols and flagged them for future dictionary expansion[cite: 1].

13. Learnings & Key Takeaways

- **Data Structure Efficiency:** Utilizing hash maps (dictionaries) allows efficient lookup operations for translation tasks[cite: 1].
- **String Tokenization:** Learned to leverage Python's built-in `.split()` method with single and double space parameters to parse nested structure representations[cite: 1].
- **Clean Code Structure:** Proper block indentation is essential in Python for managing program flow control within nested loops[cite: 1].

14. Future Enhancements

- **Graphical User Interface (GUI):** Implement a desktop interface using Python's tkinter library.
- **Audio Playback:** Add dynamic tone generation (winsound or pygame) to play dots and dashes as audible signals.
- **File I/O Integration:** Enable bulk file conversion by reading from .txt files and writing translated outputs directly to disk.
- **Expanded Symbol Mapping:** Include standard punctuation marks (period, comma, question mark) and international Morse extensions.

15. References

1. Python Software Foundation. *Python 3 Documentation - Dictionaries and Control Flow*. <https://docs.python.org/3/>
2. International Telecommunication Union (ITU). *Recommendation ITU-R M.1677-1: International Morse code*. <https://www.itu.int>
3. Source Code Reference: morse code converter.py